```
import warnings; warnings.filterwarnings("ignore")
import os, zipfile
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
import plotly.express as px
from scipy import stats
import statsmodels.api as sm
from statsmodels.formula.api import ols
from statsmodels.stats.multicomp import pairwise_tukeyhsd
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score, GridSearchCV, learning_curve
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
    confusion_matrix, ConfusionMatrixDisplay, roc_curve, precision_recall_curve, classification_report)

sns.set_theme(style="whitegrid")
pd.set_option("display.width", 200); pd.set_option("display.max_columns", 50)
os.makedirs("figs", exist_ok=True)
def H(t): print("\n" + "="*75 + "\n" + t + "\n" + "="*75)
def save(name):
    plt.savefig(f"figs/{name}.png", dpi=200, bbox_inches="tight"); plt.show()

URL = "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv"
raw = pd.read_csv(URL)

# ============================ WEEK 1 ============================
H("WEEK 1: RAW DATA INSPECTION")
print("Shape:", raw.shape)
print(raw.dtypes.to_string())
print("\nNulls (isnull) in raw:\n", raw.isnull().sum()[raw.isnull().sum() > 0])
blank = (raw["TotalCharges"].astype(str).str.strip() == "")
print("Blank-string TotalCharges count:", blank.sum())
print("Full-row duplicates:", raw.duplicated().sum(), "| Duplicate customerIDs:", raw["customerID"].duplicated().sum())

df = raw.copy()
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
print("\nNaN after to_numeric:", df["TotalCharges"].isnull().sum())
print("Rows with NaN TotalCharges (tenure, MonthlyCharges, Churn):")
print(df[df["TotalCharges"].isnull()][["customerID","tenure","MonthlyCharges","Churn"]].to_string())
print("Tenure values among those rows:", df[df["TotalCharges"].isnull()]["tenure"].unique())

df["TotalCharges"] = df["TotalCharges"].fillna(0)
df["SeniorCitizen"] = df["SeniorCitizen"].map({0:"No", 1:"Yes"})
obj_cols = df.select_dtypes("object").columns
df[obj_cols] = df[obj_cols].apply(lambda s: s.str.strip())
df["Churn_flag"] = (df["Churn"] == "Yes").astype(int)
print("\nNulls after cleaning:", df.isnull().sum().sum())
print("Unique values of categorical columns:")
for c in obj_cols.drop("customerID"):
    print(f"  {c}: {list(df[c].unique())}")

H("WEEK 1: OUTLIERS (IQR rule)")
for col in ["tenure","MonthlyCharges","TotalCharges"]:
    q1, q3 = df[col].quantile([.25,.75]); iqr = q3 - q1
    n = ((df[col] < q1-1.5*iqr) | (df[col] > q3+1.5*iqr)).sum()
    print(f"{col}: IQR outliers = {n}, skew = {df[col].skew():.3f}")

H("WEEK 1: SUMMARY STATISTICS")
print(df[["tenure","MonthlyCharges","TotalCharges"]].describe().round(2).to_string())
print("\nMedian:", df[["tenure","MonthlyCharges","TotalCharges"]].median().round(2).to_dict())
print("\nChurn counts:\n", df["Churn"].value_counts().to_string())
print("Churn %:\n", (df["Churn"].value_counts(normalize=True)*100).round(2).to_string())
print("\nCorrelation matrix:\n", df[["tenure","MonthlyCharges","TotalCharges","Churn_flag"]].corr().round(3).to_string())

H("WEEK 1: CHURN RATE (%) BY CATEGORY")
for c in ["Contract","InternetService","PaymentMethod","TechSupport","OnlineSecurity","SeniorCitizen",
          "Partner","Dependents","gender","PaperlessBilling","StreamingTV","MultipleLines"]:
    print(f"\n{c}:\n", (df.groupby(c)["Churn_flag"].mean()*100).round(2).to_string())

df.to_csv("telco_clean.csv", index=False)

# W1 figures
plt.figure(figsize=(10,4)); r2 = raw.copy(); r2["TotalCharges"] = pd.to_numeric(r2["TotalCharges"], errors="coerce")
sns.heatmap(r2.isnull(), cbar=False, yticklabels=False); plt.title("Missing Values Map (before cleaning)"); save("w1_fig1_mi
fig, ax = plt.subplots(1,3, figsize=(15,4))
for a, col in zip(ax, ["tenure","MonthlyCharges","TotalCharges"]):
    sns.histplot(df[col], kde=True, ax=a); a.set_title(f"Distribution of {col}")
```

```
    sns.histplot(df[col], kde=True, ax=ax); ax.set_title(f"Distribution of {col}")
save("w1_fig2_hist")
plt.figure(figsize=(6,5)); sns.boxplot(data=df, x="Churn", y="MonthlyCharges"); plt.title("Monthly Charges by Churn"); save(
(df.groupby("Contract")["Churn_flag"].mean()*100).plot(kind="bar", figsize=(6,4), color="#d1495b")
plt.ylabel("Churn rate (%)"); plt.title("Churn Rate by Contract Type"); plt.xticks(rotation=0); save("w1_fig4_contract")
plt.figure(figsize=(6,5)); sns.heatmap(df[["tenure","MonthlyCharges","TotalCharges","Churn_flag"]].corr(), annot=True, cmap=
plt.title("Correlation Matrix"); save("w1_fig5_corr")

# ============================ WEEK 2 ============================
H("WEEK 2: STORY NUMBERS")
COL = {"Yes":"#d1495b", "No":"#4c78a8"}
df["tenure_band"] = pd.cut(df["tenure"], [-1,6,12,24,48,72], labels=["0-6","7-12","13-24","25-48","49-72"])
t = df.groupby("tenure_band", observed=True)["Churn_flag"].mean().mul(100)
print("Churn % by tenure band:\n", t.round(2).to_string())
print("\nChurn % by contract:\n", pd.crosstab(df["Contract"], df["Churn"], normalize="index").mul(100).round(2).to_string())
pv = df.pivot_table(index="InternetService", columns="Contract", values="Churn_flag", aggfunc="mean").mul(100)
print("\nChurn % InternetService x Contract:\n", pv.round(1).to_string())
print("\nMean MonthlyCharges by churn:", df.groupby("Churn")["MonthlyCharges"].mean().round(2).to_dict())
sub = df[df["InternetService"] != "No"]
rows = []
for s in ["OnlineSecurity","TechSupport","OnlineBackup","DeviceProtection"]:
    r = sub.groupby(s)["Churn_flag"].mean().mul(100)
    rows.append({"Service":s, "Has service":r.get("Yes"), "No service":r.get("No")})
svc_df = pd.DataFrame(rows).set_index("Service"); print("\nAdd-on churn % (internet customers):\n", svc_df.round(2).to_strin

counts = df["Churn"].value_counts()
fig, ax = plt.subplots(figsize=(5,5)); ax.pie(counts, labels=counts.index, colors=[COL[i] for i in counts.index], autopct="%
ax.set_title("About 1 in 4 customers has left"); save("w2_v1_donut")
plt.figure(figsize=(7,4)); plt.plot(t.index.astype(str), t.values, marker="o", color=COL["Yes"], lw=2.5)
for x, y in zip(t.index.astype(str), t.values): plt.text(x, y+1, f"{y:.0f}%", ha="center")
plt.title("Churn is highest in the first months and falls with loyalty"); plt.ylabel("Churn rate (%)"); plt.xlabel("Months a
ct_ = pd.crosstab(df["Contract"], df["Churn"], normalize="index").mul(100)
ct_[["No","Yes"]].plot(kind="barh", stacked=True, color=[COL["No"],COL["Yes"]], figsize=(8,3.5))
plt.title("Month-to-month customers leave far more often than contracted ones"); plt.xlabel("% of customers"); save("w2_v3_c
plt.figure(figsize=(7,4))
for k, g in df.groupby("Churn"): sns.kdeplot(g["MonthlyCharges"], fill=True, alpha=.4, color=COL[k], label=f"Churn = {k}")
plt.title("Churners are concentrated at higher monthly bills"); plt.legend(); save("w2_v4_price")
plt.figure(figsize=(7,4)); sns.heatmap(pv, annot=True, fmt=".0f", cmap="Reds", cbar_kws={"label":"Churn %"})
plt.title("Highest risk: fiber optic on month-to-month"); save("w2_v5_heatmap")
svc_df.plot(kind="bar", figsize=(8,4), color=["#59a14f","#d1495b"]); plt.title("Customers with security and support services
plt.ylabel("Churn rate (%)"); plt.xticks(rotation=0); save("w2_v6_services")
plt.figure(figsize=(7,5)); sns.scatterplot(data=df, x="tenure", y="MonthlyCharges", hue="Churn", palette=COL, alpha=.5)
plt.title("Every customer: tenure vs monthly bill"); save("w2_v7_scatter")
px.scatter(df, x="tenure", y="MonthlyCharges", color="Churn", color_discrete_map=COL, opacity=.6,
           hover_data=["Contract","InternetService"]).write_html("figs/w2_v7_interactive.html")

# ============================ WEEK 3 ============================
H("WEEK 3: HYPOTHESIS TESTS (alpha = 0.05, Bonferroni = 0.0125)")
a = df.loc[df.Churn=="Yes","MonthlyCharges"]; b = df.loc[df.Churn=="No","MonthlyCharges"]
print(f"H1: mean churned = {a.mean():.2f} (sd {a.std():.2f}, n={len(a)}), retained = {b.mean():.2f} (sd {b.std():.2f}, n={le
print("Shapiro-type normality (D'Agostino) churned:", stats.normaltest(a), "retained:", stats.normaltest(b))
print("Levene:", stats.levene(a, b))
tt, p = stats.ttest_ind(a, b, equal_var=False); print(f"Welch t = {tt:.4f}, p = {p:.3e}")
diff = a.mean()-b.mean(); va, vb = a.var(ddof=1)/len(a), b.var(ddof=1)/len(b); se = np.sqrt(va+vb)
dof = (va+vb)**2/(va**2/(len(a)-1)+vb**2/(len(b)-1)); ci = (diff-stats.t.ppf(.975,dof)*se, diff+stats.t.ppf(.975,dof)*se)
sp = np.sqrt(((len(a)-1)*a.var(ddof=1)+(len(b)-1)*b.var(ddof=1))/(len(a)+len(b)-2))
print(f"Diff = {diff:.3f}, 95% CI = ({ci[0]:.3f}, {ci[1]:.3f}), Cohen's d = {diff/sp:.3f}, Welch dof = {dof:.1f}")
print("Mann-Whitney:", stats.mannwhitneyu(a, b, alternative="two-sided"))

ctab = pd.crosstab(df["Contract"], df["Churn"]); chi2, p, dofc, exp_c = stats.chi2_contingency(ctab)
print(f"\nH2: Contract x Churn\n{ctab.to_string()}\nchi2 = {chi2:.3f}, dof = {dofc}, p = {p:.3e}, min expected = {exp_c.min(
print(f"Cramer's V = {np.sqrt(chi2/(ctab.values.sum()*(min(ctab.shape)-1))):.4f}")
print("Row %:\n", ctab.div(ctab.sum(axis=1), axis=0).mul(100).round(2).to_string())

groups = [g["tenure"].values for _, g in df.groupby("Contract")]
print("\nH3: tenure by contract. Means:", df.groupby("Contract")["tenure"].mean().round(2).to_dict())
print("Levene:", stats.levene(*groups)); F, p = stats.f_oneway(*groups); print(f"ANOVA F = {F:.3f}, p = {p:.3e}")
aov = sm.stats.anova_lm(ols("tenure ~ C(Contract)", data=df).fit(), typ=2); print(aov.round(4).to_string())
print(f"eta squared = {aov['sum_sq'].iloc[0]/aov['sum_sq'].sum():.4f}")
print(pairwise_tukeyhsd(df["tenure"], df["Contract"]).summary())
print("Kruskal-Wallis:", stats.kruskal(*groups))

ct2 = pd.crosstab(df["PaymentMethod"], df["Churn"]); chi2b, pb, dofb, _ = stats.chi2_contingency(ct2)
print(f"\nH4: Payment x Churn\nchi2 = {chi2b:.3f}, dof = {dofb}, p = {pb:.3e}, Cramer's V = {np.sqrt(chi2b/(ct2.values.sum()
print("Row %:\n", ct2.div(ct2.sum(axis=1), axis=0).mul(100).round(2).to_string())

# W3 figures
plt.figure(figsize=(7,4)); sns.histplot(data=df, x="MonthlyCharges", hue="Churn", stat="density", common_norm=False, alpha=.
plt.axvline(a.mean(), color="C1", ls="--"); plt.axvline(b.mean(), color="C0", ls="--"); plt.title("Monthly charges by churn
plt.figure(figsize=(5,4)); plt.errorbar([0],[diff], yerr=[[diff-ci[0]],[ci[1]-diff]], fmt="o", capsize=8); plt.axhline(0, co
plt.xticks([]); plt.ylabel("Mean diff in monthly charge (churned - retained)"); plt.title("95% CI of the difference"); save(
resid = (ctab.values - exp_c)/np.sqrt(exp_c)
sns.heatmap(pd.DataFrame(resid, index=ctab.index, columns=ctab.columns), annot=True, fmt=".1f", cmap="coolwarm", center=0)
```

```
plt.title("Standardized residuals: contract vs churn"); save("w3_fig3_resid")
sns.boxplot(data=df, x="Contract", y="tenure"); plt.title("Tenure by contract type"); save("w3_fig4_box")

# ============================ WEEK 4 ============================
H("WEEK 4: MODELLING")
d4 = df.drop(columns=["customerID","tenure_band"]).copy()
for c in ["MultipleLines","OnlineSecurity","OnlineBackup","DeviceProtection","TechSupport","StreamingTV","StreamingMovies"]:
    d4[c] = d4[c].replace({"No internet service":"No","No phone service":"No"})
y = d4["Churn_flag"]; X = d4.drop(columns=["Churn","Churn_flag"])
num = ["tenure","MonthlyCharges","TotalCharges"]; cat = [c for c in X.columns if c not in num]
pre = ColumnTransformer([("num", StandardScaler(), num), ("cat", OneHotEncoder(drop="if_binary", handle_unknown="ignore"), c
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
print("Train/test sizes:", len(X_tr), len(X_te), "| churn rate train/test:", round(y_tr.mean(),4), round(y_te.mean(),4))
print("Baseline accuracy (predict all 'No'):", round(1-y_te.mean(),4))
models = {"Logistic Regression": LogisticRegression(max_iter=1000, class_weight="balanced"),
          "Decision Tree": DecisionTreeClassifier(max_depth=5, class_weight="balanced", random_state=42),
          "Random Forest": RandomForestClassifier(n_estimators=300, max_depth=8, class_weight="balanced", random_state=42, n
pipes = {n: Pipeline([("pre", pre), ("clf", m)]) for n, m in models.items()}
skf = StratifiedKFold(5, shuffle=True, random_state=42); rows = []
for n, p_ in pipes.items():
    cv = cross_val_score(p_, X_tr, y_tr, cv=skf, scoring="roc_auc"); p_.fit(X_tr, y_tr)
    pred = p_.predict(X_te); proba = p_.predict_proba(X_te)[:,1]
    print(f"{n}: CV AUC = {cv.mean():.3f} +/- {cv.std():.3f}")
    rows.append({"Model":n, "Accuracy":accuracy_score(y_te,pred), "Precision":precision_score(y_te,pred), "Recall":recall_sc
                 "F1":f1_score(y_te,pred), "ROC-AUC":roc_auc_score(y_te,proba), "Train acc":accuracy_score(y_tr,p_.predict(X
                 "CV AUC":cv.mean()})
res = pd.DataFrame(rows).set_index("Model").round(3); print("\n", res.to_string())
for n, p_ in pipes.items():
    tn, fp, fn, tp = confusion_matrix(y_te, p_.predict(X_te)).ravel()
    print(f"{n} confusion: TN={tn} FP={fp} FN={fn} TP={tp} | FPR={fp/(fp+tn):.3f} FNR={fn/(fn+tp):.3f}")
print("\nLogistic Regression report:\n", classification_report(y_te, pipes["Logistic Regression"].predict(X_te), target_name

lr = pipes["Logistic Regression"]; proba = lr.predict_proba(X_te)[:,1]
print("Threshold tuning (LR):")
for th in [0.3, 0.4, 0.5, 0.6, 0.7]:
    pr_ = (proba >= th).astype(int)
    print(f"  t={th}: precision={precision_score(y_te,pr_):.3f} recall={recall_score(y_te,pr_):.3f} f1={f1_score(y_te,pr_):.
coef = pd.Series(lr.named_steps["clf"].coef_[0], index=lr.named_steps["pre"].get_feature_names_out()).sort_values()
print("\nTop negative coefficients:\n", coef.head(8).round(3).to_string()); print("Top positive coefficients:\n", coef.tail(
print("Odds ratios (top 5 positive):\n", np.exp(coef.tail(5)).round(2).to_string())
rf_imp = pd.Series(pipes["Random Forest"].named_steps["clf"].feature_importances_, index=pipes["Random Forest"].named_steps[
print("\nRandom Forest top 8 importances:\n", rf_imp.head(8).round(3).to_string())
gs = GridSearchCV(lr, {"clf__C":[0.01,0.1,1,10]}, cv=skf, scoring="roc_auc").fit(X_tr, y_tr)
print("\nGridSearch best params:", gs.best_params_, "best CV AUC:", round(gs.best_score_,4), "| test AUC:", round(roc_auc_sc

fig, ax = plt.subplots(1,3, figsize=(15,4))
for a_, (n, p_) in zip(ax, pipes.items()):
    ConfusionMatrixDisplay.from_estimator(p_, X_te, y_te, ax=a_, cmap="Blues", display_labels=["Stay","Churn"], colorbar=Fal
save("w4_fig1_confusion")
plt.figure(figsize=(6,5))
for n, p_ in pipes.items():
    pb_ = p_.predict_proba(X_te)[:,1]; fpr, tpr, _ = roc_curve(y_te, pb_); plt.plot(fpr, tpr, label=f"{n} (AUC={roc_auc_scor
plt.plot([0,1],[0,1],"k--"); plt.xlabel("False positive rate"); plt.ylabel("True positive rate"); plt.legend(); plt.title("R
plt.figure(figsize=(6,5))
for n, p_ in pipes.items():
    pr_, rc_, _ = precision_recall_curve(y_te, p_.predict_proba(X_te)[:,1]); plt.plot(rc_, pr_, label=n)
plt.xlabel("Recall"); plt.ylabel("Precision"); plt.legend(); plt.title("Precision-Recall curves"); save("w4_fig3_pr")
pd.concat([coef.head(8), coef.tail(8)]).plot(kind="barh", figsize=(7,6)); plt.title("Strongest drivers (logistic regression)
sizes, trs, vas = learning_curve(pipes["Random Forest"], X_tr, y_tr, cv=5, scoring="roc_auc", train_sizes=np.linspace(.1,1,6
plt.figure(figsize=(6,4)); plt.plot(sizes, trs.mean(1), label="Train"); plt.plot(sizes, vas.mean(1), label="Validation")
plt.legend(); plt.xlabel("Training examples"); plt.ylabel("ROC-AUC"); plt.title("Learning curve (Random Forest)"); save("w4_
print("Learning curve final train/val AUC:", round(trs.mean(1)[-1],3), round(vas.mean(1)[-1],3))

# ============================ WEEK 5 ============================
H("WEEK 5: BUSINESS IMPACT NUMBERS")
ch = df[df.Churn=="Yes"]
print("Customers:", len(df), "| churned:", len(ch), f"({len(ch)/len(df)*100:.2f}%)")
print("Avg monthly charge churned / retained:", round(ch["MonthlyCharges"].mean(),2), round(df[df.Churn=="No"]["MonthlyCharg
print("Monthly revenue lost:", round(ch["MonthlyCharges"].sum(),2), "| Annual recurring revenue lost:", round(ch["MonthlyCha
for r in [0.10, 0.20, 0.30]:
    print(f"  {int(r*100)}% fewer churners -> {int(len(ch)*r)} customers, annual revenue retained = {ch['MonthlyCharges'].su
print("Share of churners on month-to-month:", round((ch["Contract"]=="Month-to-month").mean()*100,1), "%")
print("Share of churners with tenure <= 12:", round((ch["tenure"]<=12).mean()*100,1), "%")
print("Share of churners on fiber:", round((ch["InternetService"]=="Fiber optic").mean()*100,1), "%")
print("Share of churners paying by electronic check:", round((ch["PaymentMethod"]=="Electronic check").mean()*100,1), "%")
seg = df[(df.Contract=="Month-to-month") & (df.InternetService=="Fiber optic") & (df.tenure<=12)]
print(f"High-risk segment (M2M + fiber + tenure<=12): n={len(seg)}, churn rate={seg['Churn_flag'].mean()*100:.1f}%")
seg2 = df[(df.Contract=="Month-to-month") & (df.PaymentMethod=="Electronic check")]
print(f"M2M + electronic check: n={len(seg2)}, churn rate={seg2['Churn_flag'].mean()*100:.1f}%")
print("Share of customers on month-to-month:", round((df.Contract=='Month-to-month').mean()*100,1), "%")

# package
df.to_csv("figs/telco_clean.csv", index=False)
```

```
with zipfile.ZipFile("churn_figures.zip", "w") as z:
    for f in os.listdir("figs"): z.write(f"figs/{f}")
try:
    from google.colab import files; files.download("churn_figures.zip")
except Exception: print("Zip saved as churn_figures.zip in the file panel.")
print("\nDONE. Copy ALL the text output above and paste it to Claude.")
```

```
================================================================================
WEEK 1: RAW DATA INSPECTION
================================================================================
Shape: (7043, 21)
customerID           object
gender               object
SeniorCitizen         int64
Partner              object
Dependents           object
tenure                int64
PhoneService         object
MultipleLines        object
InternetService      object
OnlineSecurity       object
OnlineBackup         object
DeviceProtection     object
TechSupport          object
StreamingTV          object
StreamingMovies      object
Contract             object
PaperlessBilling     object
PaymentMethod        object
MonthlyCharges      float64
TotalCharges         object
Churn                object

Nulls (isnull) in raw:
 Series([], dtype: int64)
Blank-string TotalCharges count: 11
Full-row duplicates: 0 | Duplicate customerIDs: 0

NaN after to_numeric: 11
Rows with NaN TotalCharges (tenure, MonthlyCharges, Churn):
      customerID  tenure  MonthlyCharges Churn
488   4472-LVYGI       0           52.55    No
753   3115-CZMZD       0           20.25    No
936   5709-LVOEQ       0           80.85    No
1082  4367-NUYAO       0           25.75    No
1340  1371-DWPAZ       0           56.05    No
3331  7644-OMVMY       0           19.85    No
3826  3213-VVOLG       0           25.35    No
4380  2520-SGTTA       0           20.00    No
5218  2923-ARZLG       0           19.70    No
6670  4075-WKNIU       0           73.35    No
6754  2775-SEFEE       0           61.90    No
Tenure values among those rows: [0]

Nulls after cleaning: 0
Unique values of categorical columns:
  gender: ['Female', 'Male']
  SeniorCitizen: ['No', 'Yes']
  Partner: ['Yes', 'No']
  Dependents: ['No', 'Yes']
  PhoneService: ['No', 'Yes']
  MultipleLines: ['No phone service', 'No', 'Yes']
  InternetService: ['DSL', 'Fiber optic', 'No']
  OnlineSecurity: ['No', 'Yes', 'No internet service']
  OnlineBackup: ['Yes', 'No', 'No internet service']
  DeviceProtection: ['No', 'Yes', 'No internet service']
  TechSupport: ['No', 'Yes', 'No internet service']
  StreamingTV: ['No', 'Yes', 'No internet service']
  StreamingMovies: ['No', 'Yes', 'No internet service']
  Contract: ['Month-to-month', 'One year', 'Two year']
  PaperlessBilling: ['Yes', 'No']
  PaymentMethod: ['Electronic check', 'Mailed check', 'Bank transfer (automatic)', 'Credit card (automatic)']
  Churn: ['No', 'Yes']

================================================================================
WEEK 1: OUTLIERS (IQR rule)
================================================================================
tenure: IQR outliers = 0, skew = 0.240
MonthlyCharges: IQR outliers = 0, skew = -0.221
TotalCharges: IQR outliers = 0, skew = 0.963

================================================================================
WEEK 1: SUMMARY STATISTICS
================================================================================
        tenure  MonthlyCharges  TotalCharges
count  7043.00         7043.00       7043.00
mean     32.37           64.76       2279.73
std      24.56           30.09       2266.79
min       0.00           18.25          0.00
25%       9.00           35.50        398.55
50%      29.00           70.35       1394.55
75%      55.00           89.85       3786.60
max      72.00          118.75       8684.80

Median: {'tenure': 29.0, 'MonthlyCharges': 70.35, 'TotalCharges': 1394.55}
```

```
Churn counts:
 Churn
No     5174
Yes    1869
Churn %:
 Churn
No     73.46
Yes    26.54

Correlation matrix:
                tenure  MonthlyCharges  TotalCharges  Churn_flag
tenure           1.000           0.248         0.826      -0.352
MonthlyCharges   0.248           1.000         0.651       0.193
TotalCharges     0.826           0.651         1.000      -0.198
Churn_flag      -0.352           0.193        -0.198       1.000

================================================================================
WEEK 1: CHURN RATE (%) BY CATEGORY
================================================================================

Contract:
 Contract
Month-to-month    42.71
One year          11.27
Two year           2.83

InternetService:
 InternetService
DSL            18.96
Fiber optic    41.89
No              7.40

PaymentMethod:
 PaymentMethod
Bank transfer (automatic)    16.71
Credit card (automatic)      15.24
Electronic check             45.29
Mailed check                 19.11

TechSupport:
 TechSupport
No                     41.64
No internet service     7.40
Yes                    15.17

OnlineSecurity:
 OnlineSecurity
No                     41.77
No internet service     7.40
Yes                    14.61

SeniorCitizen:
 SeniorCitizen
No     23.61
Yes    41.68

Partner:
 Partner
No     32.96
Yes    19.66

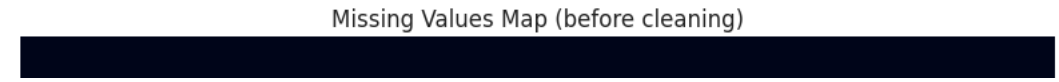
Dependents:
 Dependents
No     31.28
Yes    15.45

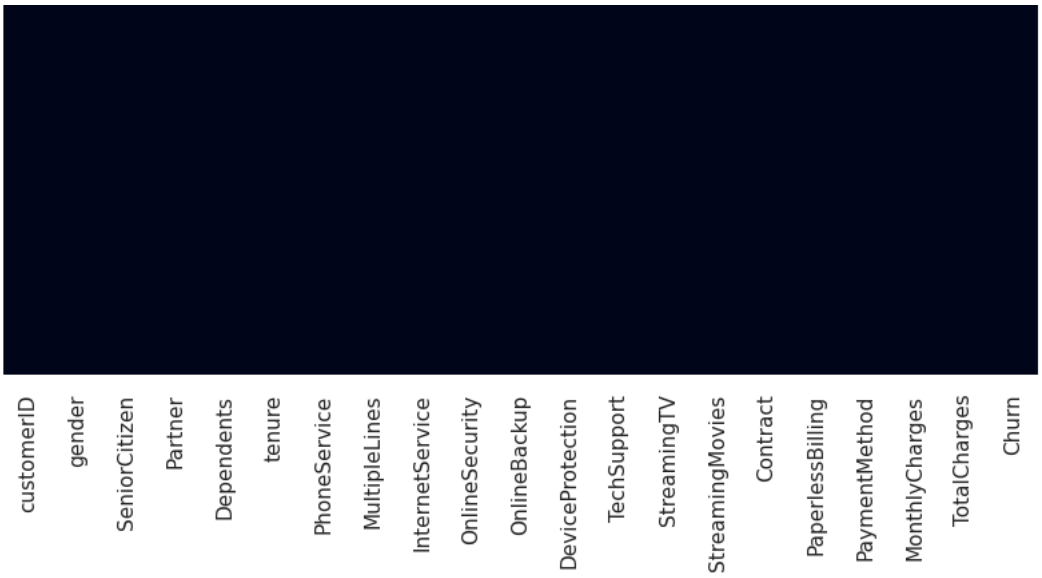
gender:
 gender
Female    26.92
Male      26.16

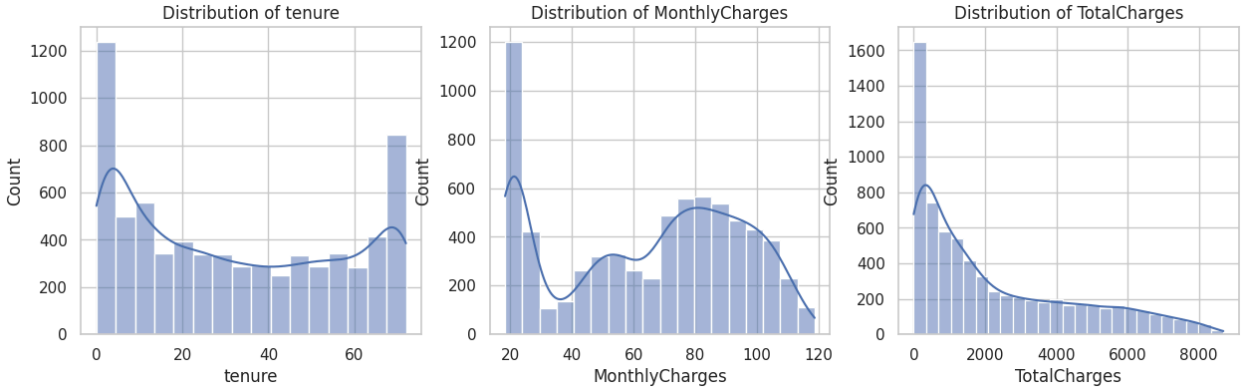
PaperlessBilling:
 PaperlessBilling
No     16.33
Yes    33.57

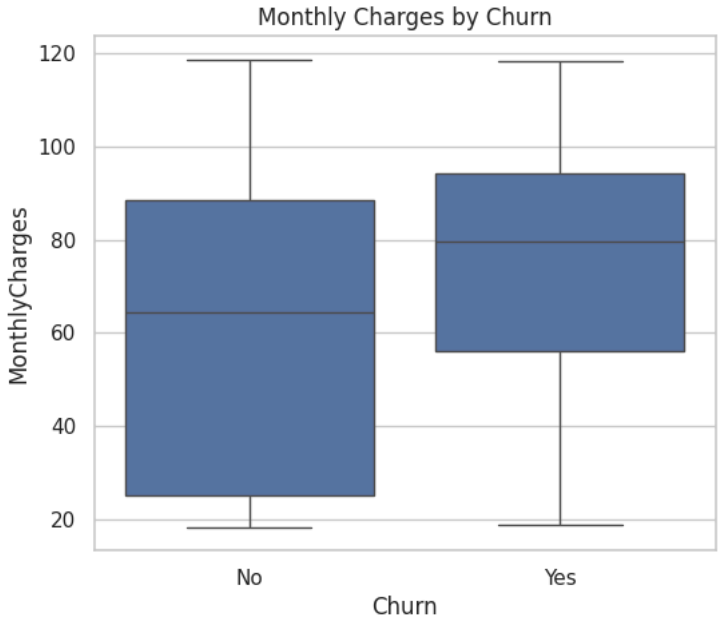
StreamingTV:
 StreamingTV
No                     33.52
No internet service     7.40
Yes                    30.07

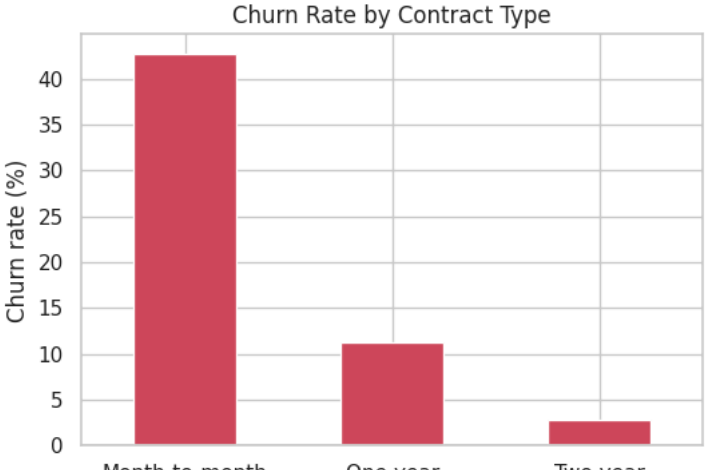
MultipleLines:
 MultipleLines
No                  25.04
No phone service    24.93
Yes                 28.61
```

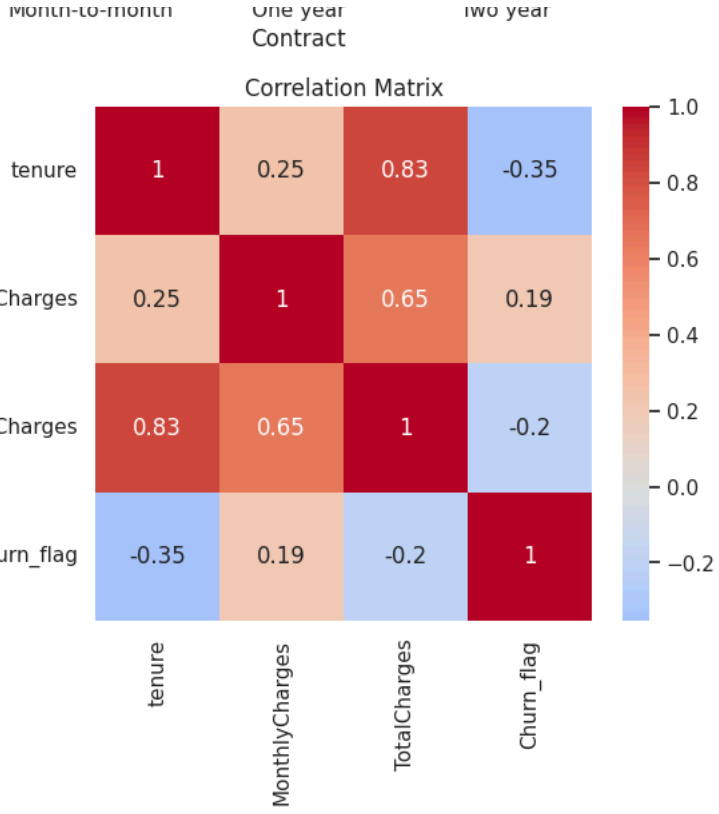



customerID
gender
SeniorCitizen
Partner
Dependents
tenure
PhoneService
MultipleLines
InternetService
OnlineSecurity
OnlineBackup
DeviceProtection
TechSupport
StreamingTV
StreamingMovies
Contract
PaperlessBilling
PaymentMethod
MonthlyCharges
TotalCharges
Churn


Distribution of tenure
Count
tenure
Distribution of MonthlyCharges
Count
MonthlyCharges
Distribution of TotalCharges
Count
TotalCharges


Monthly Charges by Churn
MonthlyCharges
No
Yes
Churn


Churn Rate by Contract Type
Churn rate (%)



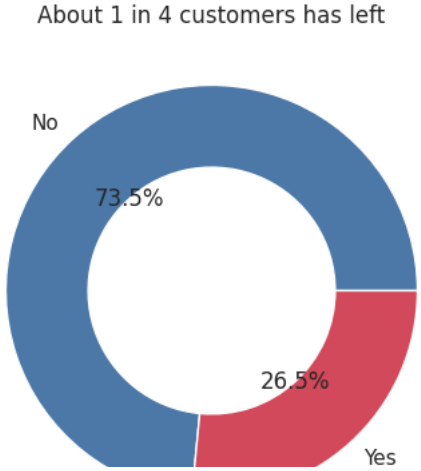

```
========================================================================
WEEK 2: STORY NUMBERS
========================================================================
Churn % by tenure band:
 tenure_band
0-6      52.94
7-12     35.89
13-24    28.71
25-48    20.39
49-72     9.51

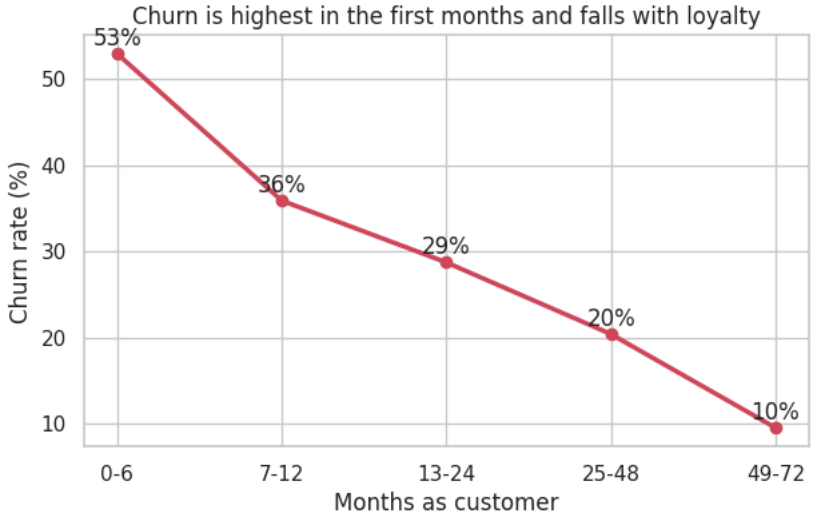
Churn % by contract:
 Churn              No    Yes
Contract
Month-to-month  57.29  42.71
One year        88.73  11.27
Two year        97.17   2.83

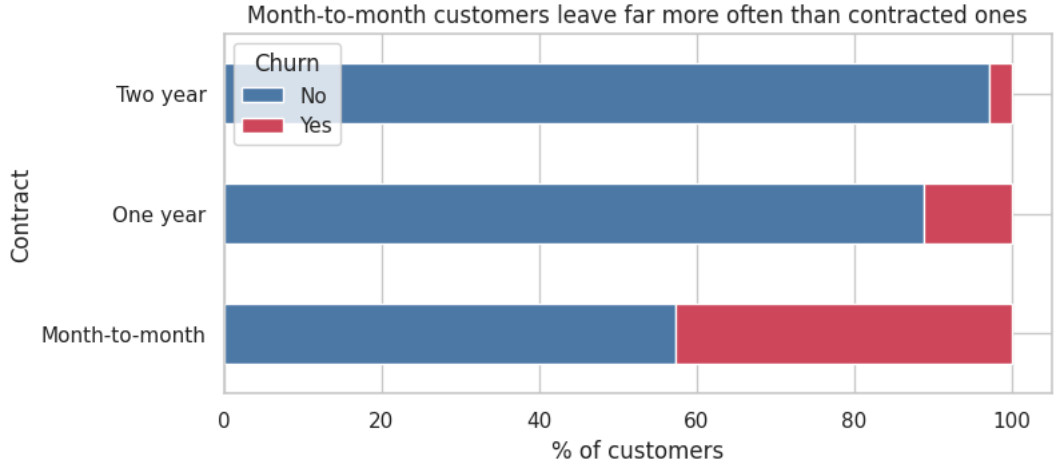
Churn % InternetService x Contract:
 Contract         Month-to-month  One year  Two year
InternetService
DSL                        32.2       9.3       1.9
Fiber optic                54.6      19.3       7.2
No                         18.9       2.5       0.8

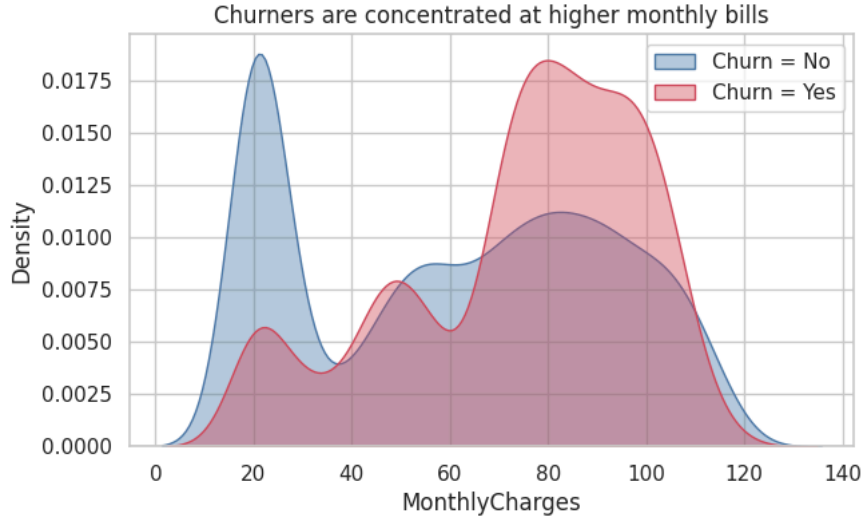
Mean MonthlyCharges by churn: {'No': 61.27, 'Yes': 74.44}

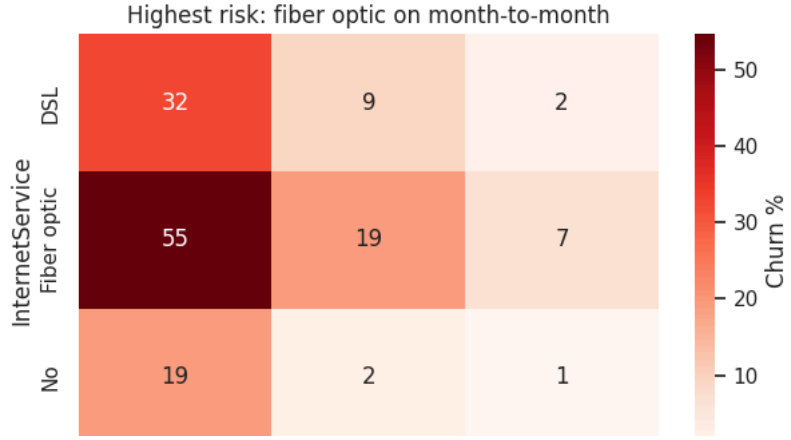
Add-on churn % (internet customers):
                  Has service  No service
Service
OnlineSecurity          14.61       41.77
TechSupport             15.17       41.64
OnlineBackup            21.53       39.93
DeviceProtection        22.50       39.13
```



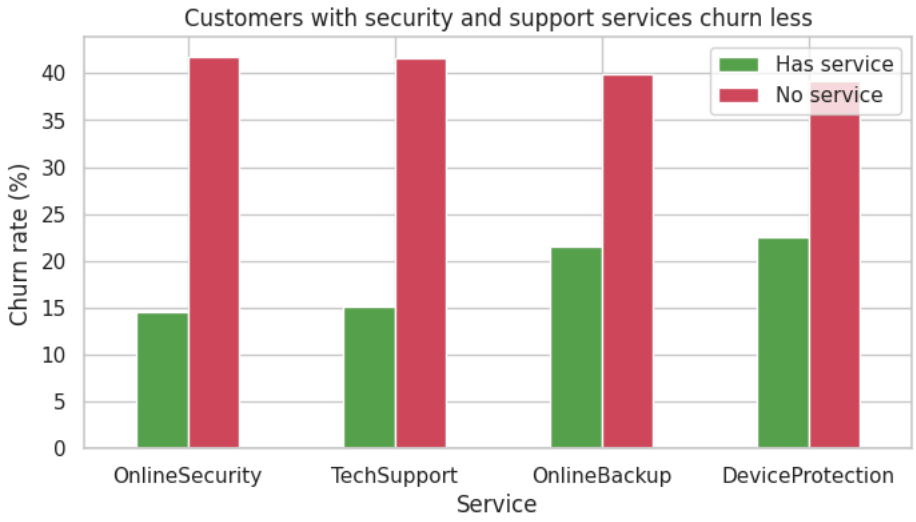

Churn is highest in the first months and falls with loyalty
Churn rate (%)
Months as customer
0-6
7-12
13-24
25-48
49-72
53%
36%
29%
20%
10%

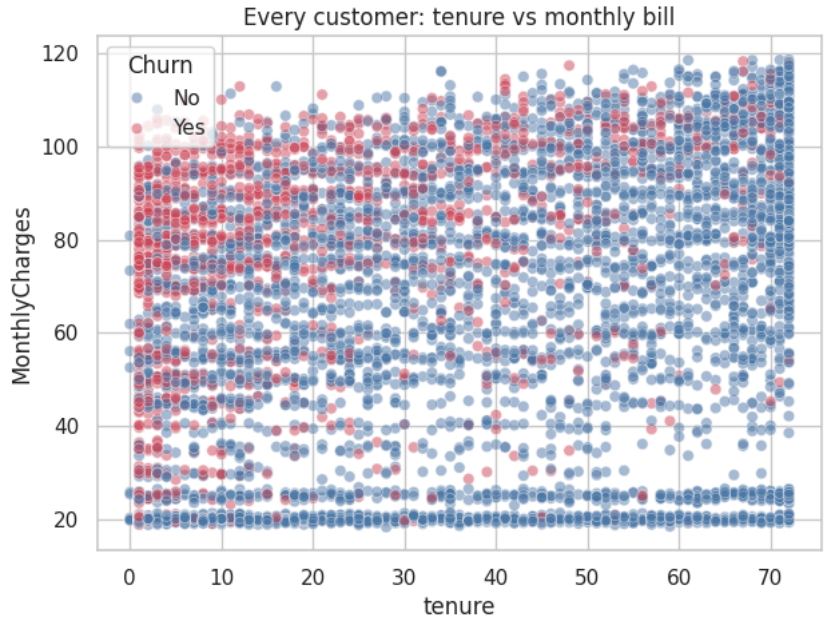

Month-to-month customers leave far more often than contracted ones
Churn
No
Yes
Contract
Two year
One year
Month-to-month
% of customers


Churners are concentrated at higher monthly bills
Churn = No
Churn = Yes
Density
MonthlyCharges


Highest risk: fiber optic on month-to-month
InternetService
DSL
Fiber optic
No
32
9
2
55
19
7
19
2
1
Churn %







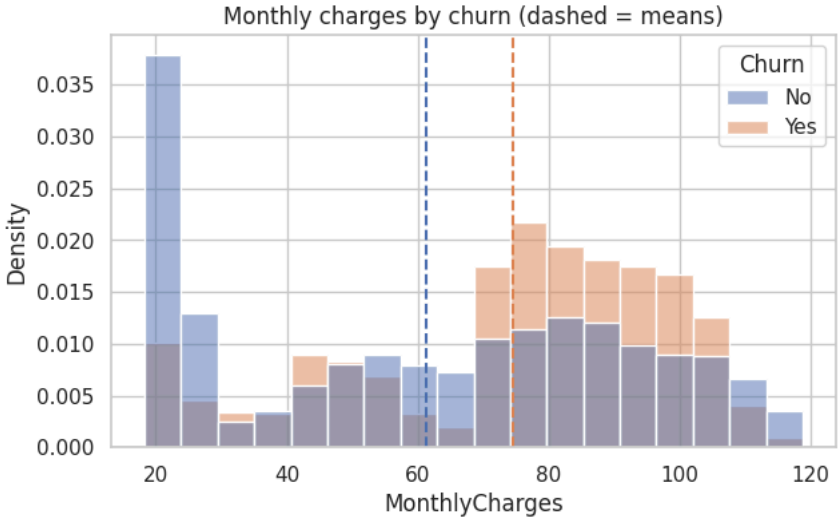

```
========================================================================
WEEK 3: HYPOTHESIS TESTS (alpha = 0.05, Bonferroni = 0.0125)
========================================================================
H1: mean churned = 74.44 (sd 24.67, n=1869), retained = 61.27 (sd 31.09, n=5174)
Shapiro-type normality (D'Agostino) churned: NormaltestResult(statistic=np.float64(150.4164578630375), pvalue=np.float64(2
Levene: LeveneResult(statistic=np.float64(361.8444859598828), pvalue=np.float64(1.0261244899421871e-78))
Welch t = 18.4075, p = 8.592e-73
Diff = 13.176, 95% CI = (11.773, 14.580), Cohen's d = 0.446, Welch dof = 4135.8
Mann-Whitney: MannwhitneyuResult(statistic=np.float64(6003125.5), pvalue=np.float64(3.311627651988585e-54))

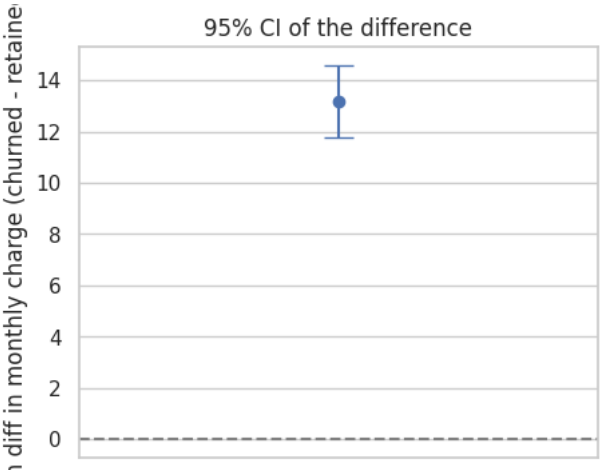
H2: Contract x Churn
Churn             No   Yes
Contract
Month-to-month  2220  1655
One year        1307   166
Two year        1647    48
chi2 = 1184.597, dof = 2, p = 5.863e-258, min expected = 390.9
Cramer's V = 0.4101
Row %:
 Churn              No    Yes
Contract
Month-to-month  57.29  42.71
One year        88.73  11.27
Two year        97.17   2.83

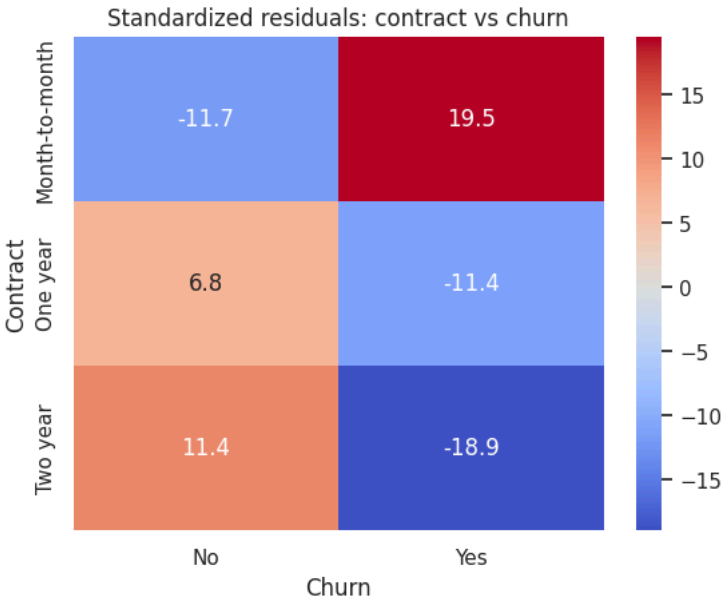
H3: tenure by contract. Means: {'Month-to-month': 18.04, 'One year': 42.04, 'Two year': 56.74}
Levene: LeveneResult(statistic=np.float64(25.489210678834855), pvalue=np.float64(9.333912186374107e-12))
ANOVA F = 2960.015, p = 0.000e+00
                  sum_sq      df          F  PR(>F)
C(Contract)  1.940226e+06     2.0  2960.0151     0.0
Residual     2.307284e+06  7040.0        NaN     NaN
eta squared = 0.4568
     Multiple Comparison of Means - Tukey HSD, FWER=0.05
============================================================
    group1      group2  meandiff p-adj  lower   upper  reject
```

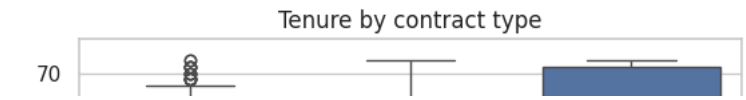
```
------------------------------------------------------------
Month-to-month One year  24.0082   0.0 22.7091 25.3072   True
Month-to-month Two year  38.6985   0.0 37.4626 39.9343   True
      One year Two year  14.6903   0.0 13.1786  16.202   True
------------------------------------------------------------
Kruskal-Wallis: KruskalResult(statistic=np.float64(3139.236066041155), pvalue=np.float64(0.0))

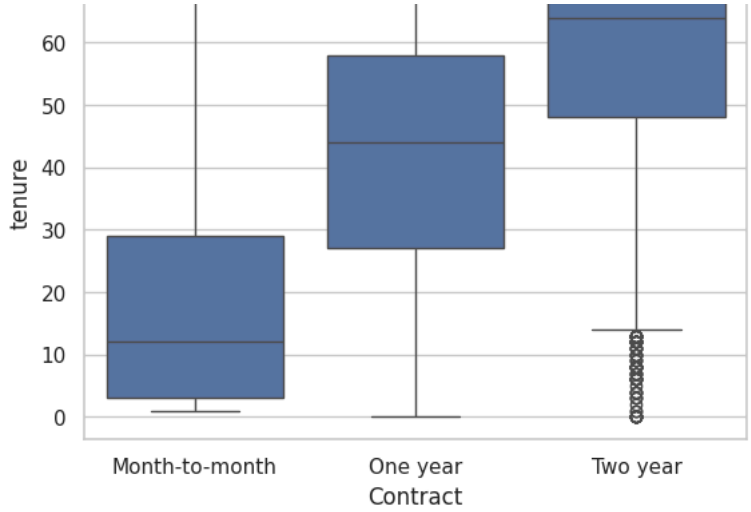
H4: Payment x Churn
chi2 = 648.142, dof = 3, p = 3.682e-140, Cramer's V = 0.3034
Row %:
 Churn                        No    Yes
PaymentMethod
Bank transfer (automatic)  83.29  16.71
Credit card (automatic)    84.76  15.24
Electronic check           54.71  45.29
Mailed check               80.89  19.11
```












```
========================================================================
WEEK 4: MODELLING
========================================================================
Train/test sizes: 5634 1409 | churn rate train/test: 0.2654 0.2654
Baseline accuracy (predict all 'No'): 0.7346
Logistic Regression: CV AUC = 0.846 +/- 0.012
Decision Tree: CV AUC = 0.828 +/- 0.013
Random Forest: CV AUC = 0.848 +/- 0.010

                     Accuracy  Precision  Recall     F1  ROC-AUC  Train acc  CV AUC
Model
Logistic Regression     0.739      0.505   0.786  0.615    0.842      0.753   0.846
Decision Tree           0.756      0.527   0.791  0.632    0.835      0.758   0.828
Random Forest           0.763      0.538   0.765  0.631    0.845      0.802   0.848
Logistic Regression confusion: TN=747 FP=288 FN=80 TP=294 | FPR=0.278 FNR=0.214
Decision Tree confusion: TN=769 FP=266 FN=78 TP=296 | FPR=0.257 FNR=0.209
Random Forest confusion: TN=789 FP=246 FN=88 TP=286 | FPR=0.238 FNR=0.235

Logistic Regression report:
              precision    recall  f1-score   support

        Stay       0.90      0.72      0.80      1035
       Churn       0.51      0.79      0.62       374

    accuracy                           0.74      1409
   macro avg       0.70      0.75      0.71      1409
weighted avg       0.80      0.74      0.75      1409

Threshold tuning (LR):
  t=0.3: precision=0.431 recall=0.928 f1=0.589
  t=0.4: precision=0.466 recall=0.866 f1=0.606
  t=0.5: precision=0.505 recall=0.786 f1=0.615
  t=0.6: precision=0.538 recall=0.703 f1=0.610
  t=0.7: precision=0.604 recall=0.604 f1=0.604

Top negative coefficients:
 cat__InternetService_No                        -1.235
num__tenure                                     -1.149
cat__Contract_Two year                          -0.886
cat__OnlineSecurity_Yes                         -0.369
num__MonthlyCharges                             -0.321
cat__PhoneService_Yes                           -0.314
cat__TechSupport_Yes                            -0.302
cat__PaymentMethod_Bank transfer (automatic)    -0.243
Top positive coefficients:
 cat__PaymentMethod_Electronic check    0.160
cat__MultipleLines_Yes                  0.288
cat__StreamingTV_Yes                    0.323
cat__PaperlessBilling_Yes               0.336
cat__StreamingMovies_Yes                0.349
num__TotalCharges                       0.483
cat__Contract_Month-to-month            0.550
cat__InternetService_Fiber optic        0.877
Odds ratios (top 5 positive):
 cat__PaperlessBilling_Yes           1.40
cat__StreamingMovies_Yes            1.42
num__TotalCharges                   1.62
cat__Contract_Month-to-month        1.73
cat__InternetService_Fiber optic    2.40

Random Forest top 8 importances:
 cat__Contract_Month-to-month           0.182
num__tenure                             0.138
num__TotalCharges                       0.110
num__MonthlyCharges                     0.099
cat__Contract_Two year                  0.095
cat__InternetService_Fiber optic        0.072
```

```
cat__PaymentMethod_Electronic check    0.052
cat__InternetService_No                0.044

GridSearch best params: {'clf__C': 10} best CV AUC: 0.8462 | test AUC: 0.8411
```

